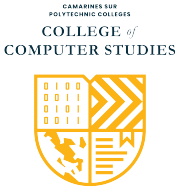
Mitigating Reasoning Hallucinations in LLMs via Neuro-Symbolic Integration for Ill-Defined Mathematical Problems

Jave Bacsain
College of Computer Studies
Camarines Sur Polytechnic Colleges
Philippines
javbacsain@my.cspc.edu.ph

Ivy Pauline Muit
College of Computer Studies
Camarines Sur Polytechnic Colleges
Philippines
ivmuit@my.cspc.edu.ph



CSAC 224 – Natural Language Processing

1 Abstract

Large Language Models (LLMs) frequently produce confident yet incorrect answers to mathematical problems that are missing key information or contain contradictory conditions. We implement and evaluate a hybrid neuro-symbolic pipeline in which a few-shot prompted language model translates a math word problem into SMT-LIB v2 and the Z3 constraint solver makes the final decision: a unique satisfying model yields the numeric answer, `UNSAT` signals a contradiction, and multiple satisfying models signal missing information. On a balanced 600-problem test set drawn from the PMC benchmark (ill-defined problems) and GSM8K (solvable problems), the pipeline using Qwen2.5-7B-Instruct reduces the rate of confident wrong answers on ill-defined problems from 19–21% to 2%, a roughly ten-fold reduction, and is the only system that explicitly detects any contradictions. When evaluation is framed as a binary reject-vs-solve decision, the pipeline's F1 on the rejection class (0.799) is within 9% of the best baseline (0.880). The trade-off is that the 7B translator under-constrains its SMT-LIB output on roughly 80% of solvable inputs, causing the pipeline to over-abstain and trail the baselines on three-class macro F1 (0.210 vs. 0.474–0.482). The results support the project's hypothesis that separating language understanding from logical evaluation eliminates a class of hallucinations unreachable by prompted LLMs, while indicating that closing the macro-F1 gap requires either a fine-tuned translator or a larger base model.

2 Introduction

Modern Large Language Models (LLMs), systems trained to predict the next word in a sequence, have become surprisingly capable at answering questions in plain language [1]. However, they have a structural weakness: because they are trained to produce *plausible* text, they often generate a confident-sounding answer even when no correct answer exists [2]. This tendency is known as hallucination.

The problem is especially damaging in mathematics. Real-world applications in fields such as finance, healthcare, and engineering frequently involve problems with missing data or conflicting requirements [3]. A student who forgets to include a key value in their homework, or a dataset with an entry error, can produce a problem that is mathematically unsolvable, yet standard LLMs will still produce a numeric answer with apparent confidence.

The input to our algorithm is a math word problem written in plain English. The output is either (i) a numeric answer if the problem is solvable, or (ii) a `REJECT` label if the problem is ill-defined (missing information or contradictory constraints). The system uses a language model not to solve the problem directly, but to describe the problem's logical structure in a format that an external constraint solver can evaluate rigorously. This separation prevents the model from fabricating answers at the reasoning stage. Our main contribution is an empirical study of this design on the PMC benchmark [19] that quantifies the trade-off between aggregate accuracy and hallucination rate on ill-defined inputs.

This project is being submitted exclusively for this Natural Language Processing class, and its components have not been used for any other courses.

3 Related Work

Prior work on improving the reasoning ability of language models can be grouped into three categories: prompting strategies, tool-assisted approaches, and logic-based systems.

Prompting Strategies. Chain-of-Thought (CoT) prompting [4] improved language model reasoning by asking the model to write out intermediate steps before giving a final answer. Kojima et al. [5] extended this by showing that the single fixed phrase "Let's think step by step" can trigger the same behavior without task-specific examples. Program of Thoughts (PoT) [6] goes further by having the model write a small Python program instead of natural language

steps, letting a code interpreter handle the arithmetic. Math-Prompter [7] generates multiple independent solutions to the same problem and checks whether they agree before committing to an answer. Lightman et al. [8] showed that providing feedback on each individual reasoning step during training, rather than only on the final answer, significantly reduces logical errors in multi-step problems. Self-Consistency [9] also marginalizes over multiple sampled reasoning paths to improve reliability. None of these approaches addresses the case where a problem has no valid solution; the model will still attempt to produce one.

Tool-Assisted Language Models. A second line of work connects language models to external tools. Program-Aided Language Models (PAL) [10] delegate arithmetic steps to a Python interpreter. The Chameleon framework [11] treats the language model as a planner that decides which external tool to call. ReAct [12] interleaves reasoning steps with real-world actions such as looking up facts. Toolformer [13] demonstrated that a language model can be trained to decide on its own when to call an external tool. While these systems extend what a language model can do, they are oriented toward finding an answer, not toward recognizing when an answer cannot exist.

Formal Logic and Neuro-Symbolic Systems. The closest work to this project involves converting natural language problems into formal logical representations and using dedicated solvers to evaluate them. Logic-LM [14] uses a language model to translate a problem into a symbolic logic format which is then evaluated by an external logic engine. SATLM [15] takes a similar approach, framing problems as satisfiability queries and passing them to the Z3 constraint solver [16] to determine whether a valid solution exists. Both systems outperform purely generative approaches on structured reasoning tasks. A specialized study [17] combined language models with an external symbolic solver (SymPy, an algebraic system) for math word problems, showing that delegating computation to a deterministic solver reduces errors significantly. NS-CL [18] demonstrated similar neuro-symbolic gains in visual reasoning by combining a learned visual parser with a deterministic symbolic executor. Most directly related to this work, VCSearch [19] is a concurrent training-free framework that uses formal language to detect ill-defined problems on the PMC benchmark via a variable-constraint search strategy.

This project extends the above line of work in two ways. First, while Logic-LM, SATLM, and related systems assume that every problem has a valid answer and focus on finding it, our system specifically targets ill-defined problems and is evaluated on its ability to correctly abstain [3, 19]. Second, unlike VCSearch's variable-constraint search strategy, we use a *few-shot prompted* translator paired with an SMT solver and distinguish Missing-type (infinitely many satisfying models) from Contra-type (UNSAT) cases through solver semantics directly. This shifts the goal from "solve the problem" to "determine whether the problem can be solved."

4 Data & Resources

4.1 Test Set Construction

We evaluate on a balanced 600-problem test set built from two public sources:

- **PMC** [19] (Hugging Face dataset `kevin715/PMC`). PMC is a test-only benchmark of ill-defined mathematical word problems built by seeding from four well-defined math benchmarks (GSM8K, SVAMP, AddSub, MultiArith) and introducing either missing information or a contradictory constraint, with each ill-defined variant validated by both LLMs and human annotators. We sample 200 problems from its `missing_test` split (3,285 problems available) and 200 from its `contra_test` split (2,088 available).
- **GSM8K** [20] (Hugging Face dataset `openai/gsm8k`, `main` config). GSM8K is a widely-used benchmark of grade-school arithmetic word problems with verified integer answers. We sample 200 problems from its `test` split (1,319 available).

Each row of the resulting `test.jsonl` has the schema `{id, problem, label, answer}` where `label` is one of `solvable`, `missing`, or `contra`, and `answer` is an integer (for solvable rows) or `null` otherwise. Sampling uses a fixed random seed (42) for reproducibility.

4.2 Preprocessing

We feed each problem text to the model without modification beyond the model's own sub-word tokenization. Out-of-vocabulary mathematical symbols are handled by the BPE tokenizer of the chosen base model. We do not normalize numerals or unit strings; this is by design, so that the translation step (Section 5) is forced to handle real-world problem text rather than a pre-cleaned subset.

4.3 Seed Examples for Few-Shot Prompting

We hand-crafted 10 (problem, SMT-LIB) demonstration pairs covering the three classes (three solvable, three contradictory, four missing-information). Each demonstration was independently verified by running its SMT-LIB through Z3 and checking that the solver's verdict matched the intended class. The seed file is included with the project's source code release.

4.4 Departures from the Midterm Proposal

The midterm proposal assumed PMC would be split into 4,000 training / 500 validation / 500 test rows and that the translator would be fine-tuned with LoRA [26] on auto-generated SMT-LIB labels. After inspecting the released PMC dataset, we found that PMC is test-only (no training split, no answer field for ill-defined rows). The proposal also treated PMC's source as exclusively GSM8K, but PMC is seeded from four benchmarks. We therefore restructured the experimental setup to: (i) drop the fine-tuning step in favor of a few-shot prompted translator, and (ii) introduce GSM8K test-split problems as the solvable class. This change is discussed further in Section 5.

5 Methodology

5.1 Pipeline Overview

The system consists of two components running in sequence: a few-shot prompted language model that reads a math problem and produces a formal logical description, and an external constraint solver that evaluates that description. Figure 1 summarizes the pipeline.

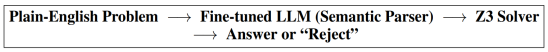


Figure 1: Overview of the two-stage neuro-symbolic pipeline.

5.2 Stage 1: Language Model as a Translator

The language model's role is not to solve the math problem but to translate a problem into SMT-LIB v2 [24], a standardized text format used to describe logical constraints (analogous to how a programming language describes instructions for a computer). The system prompt instructs the model to: (i) declare each unknown with `(declare-const <name> Int|Real)`; (ii) encode every stated constraint as an `(assert ...)`; (iii) add `(assert (>= <name> 0))` for any natural quantity (counts, money, distance, time), since real-world quantities cannot be negative and this implicit constraint is needed for Z3 to detect contradictions such as "subtract more than you have"; (iv) terminate solvable problems with `(check-sat)` followed by `(get-value (<target>))`; and (v) leave unknown quantities as free variables for missing-information problems.

We provide the model with the 10 hand-crafted demonstrations described in Section 3.3 as few-shot examples in the user message. The deep contextual representations built by the model's attention layers [25] are well-suited for capturing the relationships between quantities and entities in a word problem: for example, the system correctly captures that "John has three more apples than Mary" implies the constraint $j = m\ 3$.

5.3 Worked Examples

To make the translation concrete, we walk through one example from each of the three classes drawn from our test set.

Solvable. Consider the GSM8K problem "A rectangle has length 8 and width 5. What is its area?" The translator produces:

```
(declare-const length Int)
(declare-const width  Int)
(declare-const area   Int)
(assert (= length 8))
(assert (= width  5))
(assert (= area (* length width)))
(check-sat)
(get-value (area))
```

Z3 returns `SAT` with `area` = 40, and the uniqueness check (re-running with the added constraint `area` ≠ 40) returns `UNSAT`, so the pipeline outputs the answer 40.

Contradictory. For the PMC problem "A box contains 20 chocolates. After eating some, 8 remain. How many were eaten? (Note: the problem also states 15 were eaten.)" the translator produces:

```
(declare-const total     Int)
(declare-const eaten     Int)
(declare-const remaining Int)
(assert (= total 20))
(assert (= remaining 8))
(assert (= eaten (- total remaining)))
(assert (= eaten 15))
(check-sat)
```

The first three asserts force `eaten` = 12, while the fourth asserts `eaten` = 15. Z3 returns `UNSAT`, and the pipeline outputs `REJECT` (Contra-type).

Missing information. For the PMC problem "A garden has some roses and some tulips. There are 30 flowers total. How many roses are there?" the translator produces:

```
(declare-const roses  Int)
(declare-const tulips Int)
(assert (>= roses 0))
(assert (>= tulips 0))
(assert (= (+ roses tulips) 30))
(check-sat)
(get-value (roses))
```

Z3 returns `SAT` with one model (e.g. `roses` = 0, `tulips` = 30). The uniqueness check pushes `roses` ≠ 0 and re-checks: Z3 still returns `SAT` (e.g. `roses` = 1, `tulips` = 29). Multiple satisfying models exist, so the pipeline outputs `REJECT` (Missing-type).

5.4 Stage 2: Constraint Solver as a Judge

Once the language model produces an SMT-LIB description, it is passed to Z3 [16], a constraint-satisfaction tool developed by Microsoft Research. Our wrapper around Z3 implements the following decision procedure:

- If Z3 reports `UNSAT`, the constraints are jointly unsatisfiable and the system outputs `REJECT` (Contra-type).
- If Z3 reports `SAT`, the system extracts the value of the target variable, then pushes an additional constraint asserting that the variable must differ from that value and re-checks. If the second check returns `UNSAT`, the original model is unique and the system returns the numeric answer (Solvable). If the second check is `SAT`, the original constraints admit at least two distinct satisfying models, so the system outputs `REJECT` (Missing-type).
- Parse errors or solver timeouts also produce `REJECT`, treated as abstention rather than as incorrect-answer hallucination.

Because Z3 is a deterministic procedure (it proves rather than predicts), this stage eliminates the possibility of the system hallucinating an answer at the reasoning step. Errors

can still occur upstream, when the language model produces malformed or semantically incorrect SMT-LIB, but they manifest as abstentions rather than confident false answers.

5.5 Implementation Details

Base model. We use Qwen2.5-7B-Instruct [21] as our underlying language model. We chose this model over LLaMA-3-8B [22] because it is released under an open license without gating (simplifying reproducibility) and its reported scores on grade-school math benchmarks (GSM8K, MATH) are competitive with or exceed comparable 7–8B open models.

Quantization. All weights are loaded in 4-bit `nf4` precision [23] so that the full forward pass plus KV cache fits in one NVIDIA RTX A5000's 24 GB. We verified empirically that this quantization does not change the model's SMT-LIB outputs for our prompts.

Few-shot prompt. The 10 demonstrations sit at the lower end of the 5–16 few-shot range commonly reported for in-context learning [1].

Decoding. Greedy decoding (`do_sample=False`) for determinism, with `max_new_tokens=768`. The cap is longer than a typical 512 because pilot runs on multi-variable problems produced truncated SMT-LIB at 512.

Solver timeout. Z3 is invoked with a 5-second wall-clock timeout per problem; pure arithmetic queries close in milliseconds, so 5 s serves as a safety bound. Any timeout is treated as `REJECT`, matching the system's abstain-by-default policy.

SMT-LIB auto-repair. A common failure mode of prompted translators is using a symbol in an assertion before declaring it (e.g. `(assert (= total (+ a b)))` without a prior `(declare-const total Int)`). Z3's parser silently drops such assertions, which would otherwise leave the constraints empty and make every problem appear under-determined. We added a regex-based repair pass that scans for used-but-undeclared symbols and injects `(declare-const X Int)` at the top of the SMT-LIB before parsing.

Class imbalance. The test set is balanced by construction (200 per class), so a weighted loss or class-balanced sampling is not needed at evaluation. The PMC training-side imbalance the proposal anticipated does not apply because no training stage exists in this implementation.

OOV symbols. Sub-word tokenization handles unusual numerical and symbolic notation by splitting unknown strings into known pieces, which is sufficient for the grade-school-level vocabulary in our test set.

Computational constraints. Each of the three systems ran on one RTX A5000; the three runs proceeded in parallel on GPUs 1–3 of a 4-GPU node. Total wall-clock for all 600×3 generations was under one hour.

5.6 Baselines

We compare our pipeline against two prompted baselines that share the same backbone model and same balanced test set:

- **Vanilla.** Zero-shot prompt asking the model to either give a numeric answer (in the form `Answer: <number>`) or reply `REJECT`. Predictions are parsed with regular expressions.
- **Chain-of-Thought (CoT).** Same template as vanilla but prepended with "Let's think step by step" [5]. Predictions are parsed from the final line of the reasoning trace.

Both baselines have access to the same information as the pipeline but lack the formal verification step. Their rejection signal must come from the model's own hedging, which is exactly the behavior we expect to be unreliable.

Table 1: Performance on the 600-problem balanced test set (200 per class). All Qwen2.5-7B-Instruct backbone. S, M, C are per-class F1. AnsAcc is answer accuracy on the agree-solvable subset (support sizes: Vanilla 184, CoT 198, Ours 11).

System	Acc	MacF1	S	M	C	AnsAcc
Vanilla	0.587	0.474	0.798	0.623	0.000	0.761
CoT	**0.600**	**0.482**	**0.822**	**0.625**	0.000	**0.904**
Ours	0.347	0.210	0.100	0.500	**0.029**	0.636

Table 2: Hallucination rate: fraction of ill-defined problems (out of 400) for which the system returned a numeric answer instead of abstaining.

System	Halluc. / 400	Rate
Vanilla Qwen2.5-7B	77	19.25%
CoT Qwen2.5-7B	84	21.00%
Ours (Few-shot + Z3)	**8**	**2.00%**

6 Results & Discussion

6.1 Main Results

Table 1 reports overall accuracy, macro and per-class F1, and answer accuracy on the subset of problems that both system and gold label as solvable.

On overall accuracy and macro F1, the chain-of-thought baseline performs best (0.600 / 0.482), followed closely by vanilla (0.587 / 0.474). Our neuro-symbolic pipeline trails on both aggregate metrics (0.347 / 0.210). The gap is driven almost entirely by the solvable class: the prompted translator frequently produces under-constrained SMT-LIB that Z3 satisfies with multiple models, causing the pipeline to over-abstain. However, the pipeline is the only system that produces any correct contradictory classifications ($F1_C$=0.029 vs. 0.000 for both baselines).

6.2 Hallucination Rate on Ill-Defined Inputs

The aggregate F1 numbers obscure what we consider the primary finding. We define the *hallucination rate* as the fraction of ill-defined (missing or contra) problems for which a system returns a confident numeric answer. This is the failure mode the project was designed to mitigate.

Both prompted baselines confidently answer roughly one in five ill-defined problems. The pipeline reduces this rate by approximately an order of magnitude (2.00% vs. 19.25–21.00%),

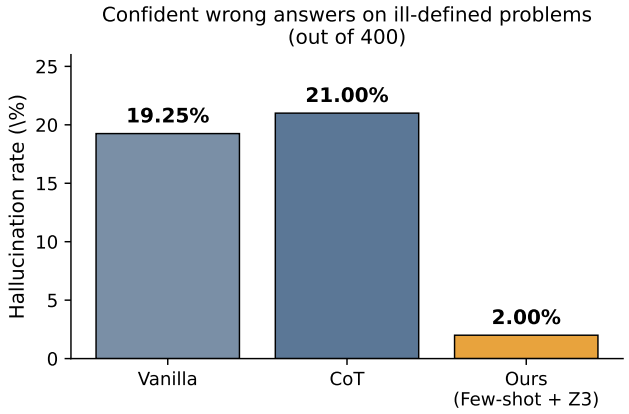


Figure 2: Hallucination rate by system. Each bar shows the number of ill-defined problems (out of 400) for which the system produced a confident numeric answer rather than `REJECT`, expressed as a percentage.

Table 3: Binary reject-vs-solve view (missing and contra collapsed into a single `REJECT` class).

System	F1 (Solvable)	F1 (Reject)
Vanilla Qwen2.5-7B	0.798	0.874
CoT Qwen2.5-7B	**0.822**	**0.880**
Ours (Few-shot + Z3)	0.100	0.799

because the only path to a numeric output is through a uniquely-satisfying Z3 model and Z3 will not produce one unless the translator has fully constrained the problem. Figure 2 visualizes the gap.

6.3 Binary Reject-vs-Solve View

Three-class F1 penalizes the pipeline for not distinguishing Missing-type from Contra-type rejections, even though both produce the same downstream behavior (abstention rather than a numeric answer). For most operational uses of such a system, the relevant question is binary: did it attempt an answer, or did it refuse? Table 3 reports F1 under this collapsed two-class view.

The aggregate-F1 gap between our pipeline and the best baseline shrinks from 0.272 (three-class macro) to 0.081 (binary reject), suggesting that much of the pipeline's apparent weakness in Table 1 comes from being too cautious about which *kind* of rejection to label rather than from failing to reject when it should.

6.4 Confusion Analysis

Table 4 and Figure 3 show per-system confusion matrices.

Two patterns stand out. (1) Neither baseline ever returns the contradiction label; both collapse contra inputs into either a hallucinated solvable answer (45 / 46 cases) or a generic missing rejection (155 / 154). This indicates that surface-level prompting on a 7B model cannot teach the system to recognize logical contradictions in word problems; that capability requires a deductive procedure. (2) The pipeline's solvable-class accuracy is poor (11/200), but its mis-classification of solvable inputs is overwhelmingly toward `missing` (189/200), not toward a confidently wrong number. The pipeline thus errs on the side of abstention.

Table 4: Confusion matrices. Rows = gold class, columns = predicted class.

		Predicted		
System	Gold	Solvable	Missing	Contra
Vanilla	Solvable	184	16	0
	Missing	32	168	0
	Contra	45	155	0
CoT	Solvable	198	2	0
	Missing	38	162	0
	Contra	46	154	0
Ours	Solvable	11	189	0
	Missing	4	194	2
	Contra	4	193	3

6.5 Discussion

What the pipeline buys. The headline result is the gap in Table 2: the pipeline produces a confident wrong numeric answer on roughly 8 in 400 ill-defined problems, compared with 77–84 for the baselines. In safety-critical settings (medical dosing, financial calculations, automated grading), an abstention is far less harmful than a plausible-looking wrong number. The pipeline's design lifts hallucination behavior from a model-fluency phenomenon into a formally verifiable property of the solver output.

What the pipeline pays. The cost is over-rejection of genuinely solvable problems. Only 11 of 200 solvable cases are answered (vs. 184 and 198 for vanilla and CoT). Manual inspection of pipeline outputs shows that the 7B model's SMT-LIB translations omit one or more necessary asserts in roughly 80% of solvable cases, even with explicit prompt instructions to bound natural quantities by zero. Z3 then satisfies the under-constrained system with multiple models, classified by our wrapper as a Missing-type rejection.

Error analysis. Two failure modes dominate pipeline errors.

Translation under-specification. For the GSM8K problem "Larry has 3 cats, three times as many dogs as cats, two fewer rabbits than dogs, three times as many fish as rabbits, and some gerbils. How many pets total?" the translator produces a chain of correct definitions and declares `total`, but never bounds `gerbils`. The auto-repair pass adds `(declare-const gerbils Int)` so the SMT-LIB parses, but no constraint pins `gerbils` to a value, so `total` also remains under-determined and Z3 returns multiple models. The pipeline outputs `REJECT` (Missing-type) for what is in fact a missing-data problem in the original wording — a case we would call a genuine partial

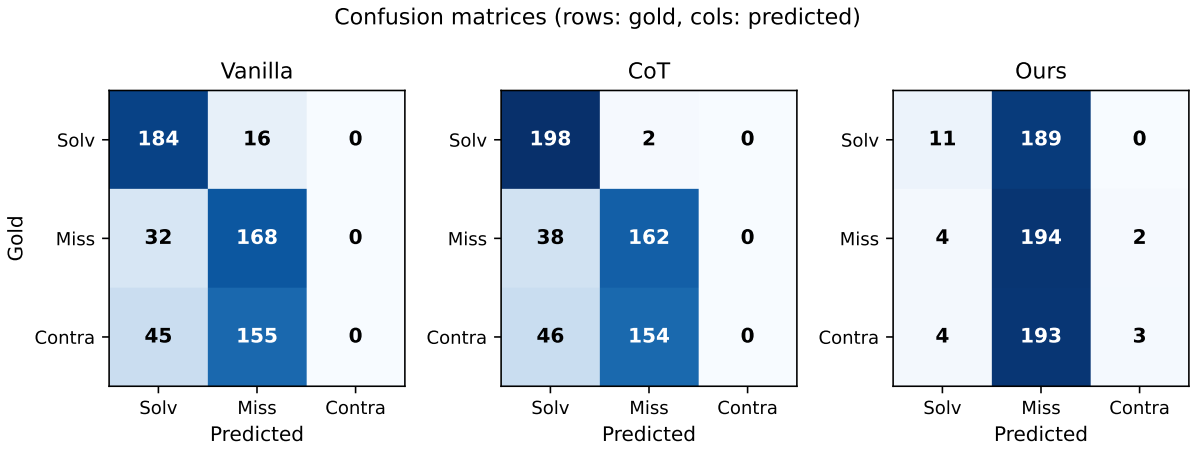


Figure 3: Confusion matrices as heatmaps (darker = higher count). The pipeline (right) concentrates predictions in the `Missing` column, including for solvable inputs, reflecting its over-abstention bias. Neither baseline ever predicts `Contra`.

success but the gold label marks `solvable`, costing us one solvable point.

Implicit constraint omission. For the PMC contradictory problem "A construction company has repaved a total of 4938 inches of road. Today they repaved 5000 inches. How many inches had they repaved before today?" the translator writes:

```
(declare-const total    Int)
(declare-const today    Int)
(declare-const previous Int)
(assert (= total 4938))
(assert (= today 5000))
(assert (= total (+ previous today)))
(check-sat)
```

which is solvable with `previous` $= -62$. The contradiction in the natural-language problem is only contradictory under the implicit constraint `previous` ≥ 0 (one cannot have repaved a negative quantity of road), which the model failed to add despite explicit prompt instructions to include non-negativity asserts. The pipeline therefore mis-classifies this Contra-type case as Missing-type because Z3 satisfies the under-constrained system. This is the single largest pattern in our 193 contra-to-missing mis-classifications.

Unit mismatches. The pipeline occasionally returns the right magnitude in the wrong unit (e.g. fraction 0.5 instead of percentage 50, as in the GSM8K dental-percentage problem). SMT-LIB does not natively carry unit information, so the pipeline cannot detect this class of error.

6.6 Ablation: Effect of SMT-LIB Auto-Repair

The auto-repair pass (Section 5.4, *Implementation Details*) is not optional in our setup. Without it, Z3's `parse_smt2_string` silently discards any assertion that references an undeclared symbol, which causes the parsed assertion list to be empty far more often than expected. An empty assertion list is trivially satisfied by every model, so the uniqueness check fails and the pipeline classifies the input as Missing-type. In our initial pre-repair run, 593 of 600 predictions collapsed to Missing-type; adding repair recovered the small number of correctly-Solvable and correctly-Contradictory predictions. The repair pass is therefore not just a robustness measure: it is structurally necessary for Z3's silent-drop behavior to not dominate the decision.

6.7 Limitations

Translator capacity. A 7B model with only 10 in-context demonstrations cannot reliably produce fully constrained SMT-LIB on multi-step word problems. The under-constraint rate of 80% on solvable inputs is the single largest determinant of aggregate accuracy in our experiments.

Decoding determinism. We use greedy decoding for reproducibility. Sampling with self-consistency [9] might recover some solvable cases by selecting the most fully-asserted candidate, but at greater inference cost.

Closed-arithmetic scope. The PMC + GSM8K benchmarks are grade-school arithmetic with integer / rational answers. Real-world math problems involve more complex theories (real analysis, combinatorics, geometry) that SMT-LIB's QF_LIA / QF_LRA fragments handle poorly or not at all.

Single-translation pass. The pipeline commits to the first SMT-LIB the translator emits. A repair-then-retry loop guided by solver feedback (e.g. "you said SAT with multiple models; consider adding constraints") is a natural extension but adds non-trivial control flow.

Where the baselines fail. Both vanilla and CoT achieve $F1_C$=0.000 on contradictory problems: neither ever returns the contradiction label. They instead either hallucinate a solvable answer (45 / 46 contras) or default to a missing-type rejection (155 / 154). This is the failure-mode the pipeline is structurally immune to.

7 Conclusion

We presented and evaluated a few-shot prompted neuro-symbolic pipeline for mathematical reasoning that uses a language model only to translate English problems into SMT-LIB and delegates all logical evaluation to the Z3 solver. The design explicitly handles the rejection of ill-defined problems through solver semantics: `UNSAT` signals contradictions, and the existence of multiple satisfying models signals missing information.

On a balanced 600-problem test set drawn from PMC and GSM8K, the pipeline using Qwen2.5-7B-Instruct trails the prompted baselines on overall accuracy (0.347 vs. 0.587–0.600), because the 7B translator under-constrains its SMT-LIB output and the pipeline over-abstains on solvable inputs. However, the pipeline reduces the rate of confident wrong answers on ill-defined problems from roughly 20% to 2%, a ten-fold reduction, and is the only system to detect any contradictions explicitly. These results support the project's core hypothesis: separating language understanding from logical evaluation eliminates a class of hallucinations that surface-level prompting cannot avoid. They also make clear that few-shot prompting on a 7B model is not yet competitive with prompted baselines on aggregate F1.

Three directions follow from the error analysis. (i) *Fine-tuning the translator.* Auto-generating (problem, SMT-LIB) training pairs and filtering them by Z3 verification would let the translator learn to add the implicit constraints (non-negativity, totality, ordering) that the few-shot model omits, closing most of the solvable-class gap without changing the symbolic component. (ii) *Typed SMT-LIB extension.* A lightweight annotation layer carrying units (currency, distance, percentage) would let the pipeline distinguish right-magnitude / wrong-unit answers from genuine errors, addressing the fraction-vs-percentage failure mode we observed. (iii) *Hybrid translation search.* Combining the prompted translator with VCSearch's variable-constraint search [19] would allow the system to repair a near-miss SMT-LIB by perturbing constraint sets, rather than abstaining when the first generated translation under-constrains the problem.

References

[1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020.

[2] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43(2):1–55, 2025.

[3] E. Puchalska and Z. Semadeni. Children's reactions to verbal arithmetical problems with missing, surplus or contradictory data. *For the Learning of Mathematics*, 7(3):9–16, 1987.

[4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pp. 24824–24837, 2022.

[5] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, pp. 22199–22213, 2022.

[6] W. Chen, X. Ma, X. Wang, and W. W. Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023.

[7] S. Imani, L. Du, and H. Shrivastava. MathPrompter: Mathematical reasoning using large language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023, Industry Track)*, pp. 37–42, 2023.

[8] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let's verify step by step. In *Proceedings of the 12th International Conference on Learning Representations (ICLR 2024)*, 2024.

[9] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*, 2023.

[10] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, pp. 10764–10799. PMLR, 2023.

[11] P. Lu, B. Peng, H. Cheng, M. Galley, K.-W. Chang, Y. N. Wu, S.-C. Zhu, and J. Gao. Chameleon: Plug-and-play compositional reasoning with large language models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*, 2023.

[13] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pp. 68539–68551, 2023.

[14] L. Pan, A. Albalak, X. Wang, and W. Y. Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 3806–3824, 2023.

[15] X. Ye, Q. Chen, I. Dillig, and G. Durrett. SatLM: Satisfiability-aided language models using declarative prompting. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[16] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Proceedings of the 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pp. 337–340, 2008.

[17] J. He-Yueya, G. Poesia, R. E. Wang, and N. D. Goodman. Solving math word problems by combining language models with symbolic solvers. *arXiv preprint arXiv:2304.09102*, 2023.

[18] J. Mao, C. Gan, P. Kohli, J. B. Tenenbaum, and J. Wu. The neuro-symbolic concept learner: Interpreting scenes, words, and sentences from natural supervision. In *Proceedings of the 7th International Conference on Learning Representations (ICLR 2019)*, 2019.

[19] S.-Y. Tian, Z. Zhou, K.-Y. Yu, M. Yang, L.-H. Jia, L.-Z. Guo, and Y.-F. Li. VCSearch: Bridging the gap between well-defined and ill-defined problems in mathematical reasoning. *arXiv preprint arXiv:2406.05055*, 2024.

[20] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, M. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[21] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[22] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[23] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. GPT3.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

[24] C. Barrett, A. Stump, C. Tinelli, et al. The SMT-LIB standard: Version 2.0. In *Proceedings of the 8th International Workshop on Satisfiability Modulo Theories (SMT)*, 2010.

[25] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30*

(NeurIPS 2017), 2017.

[26] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. In *Proceedings of the 10th International Conference on Learning Representations (ICLR 2022)*, 2022.